3.12 补码乘法的硬件逻辑实现——Booth算法

考纲定位：计算机组成原理 · 第 3 章 运算方法（3.12） | 重要度 ★★★★★ | 常见考法：大题（Booth 重编码、加减选择、算术右移、结果符号）

一句话核心：Booth 算法把补码乘法转成“看乘数相邻两位，决定加/减被乘数，再算术右移”

一、补码一位乘法运算推导

1.1 被乘数符号任意，乘数符号为正

被乘数X $[X]_{补}=X_0.X_1X_2\cdots X_n$　　**乘数Y** $[Y]_{补}=Y_0.Y_1Y_2\cdots Y_n$

被乘数X的符号任意，乘数Y的符号为正

根据补码定义可得： $[X]_{补}=2+X=2^{n+1}+X\ \ (mod\ 2)$　　$[Y]_{补}=0.Y_1Y_2\cdots Y_n=Y$

$$[X]_{补}\times[Y]_{补}=[X]_{补}\times Y=(2^{n+1}+X)\times Y=2^{n+1}\times Y+X\times Y$$

$$=2^{n+1}\times 0.Y_1Y_2\cdots Y_n+X\times Y$$

$$=2\times\boxed{2^n\times 0.Y_1Y_2\cdots Y_n}+X\times Y=2\times\boxed{Y_1Y_2\cdots Y_n}+X\times Y$$

将$0.Y_1Y_2\cdots Y_n$的小数点右移n位，得到整数$Y_1Y_2\cdots Y_n$

根据模数性质，$2\times Y_1Y_2\cdots Y_n=2$，因此上式可写为$2+X\times Y$

$$[X]_{补}\times[Y]_{补}=\boxed{2+X\times Y}=\boxed{[X\times Y]_{补}}=[X]_{补}\times Y$$

补码定义

1.2 被乘数符号任意，乘数符号为负

被乘数X $[X]_{补}=X_0.X_1X_2\cdots X_n$　　**乘数Y** $[Y]_{补}=Y_0.Y_1Y_2\cdots Y_n$

被乘数X的符号任意，乘数Y的符号为正　　$[X\times Y]_{补}=[X]_{补}\times Y$　　❶

被乘数X的符号任意，乘数Y的符号为负

根据补码定义可得： $[Y]_{补}=2+Y=1.Y_1Y_2\cdots Y_n\ \ (mod\ 2)$

$$Y=[Y]_{补}-2=1.Y_1Y_2\cdots Y_n-2=1+0.Y_1Y_2\cdots Y_n-2=0.Y_1Y_2\cdots Y_n-1$$

$X\times Y=X\times(0.Y_1Y_2\cdots Y_n-1)=X\times 0.Y_1Y_2\cdots Y_n-X$　**对该式两边同时求补可得：**

$[X\times Y]_{补}=[X\times 0.Y_1Y_2\cdots Y_n-X]_{补}$　　**根据补码减法公式可得：**

$=[X\times 0.Y_1Y_2\cdots Y_n]_{补}-[X]_{补}$　　**根据上面的式❶可得：**

$=[X]_{补}\times 0.Y_1Y_2\cdots Y_n-[X]_{补}$

1.3 合并——补码一位乘法运算

设法将上面的两个式子综合成一个式子，即补码一位乘法运算的公式

符号位参与运算 $[X \times Y]_{补} = [X]_{补} \times 0.Y_1Y_2 \cdots Y_n - [X]_{补} \times Y_0$ **（引入了乘数Y的符号位Y_0）**

当乘数Y的符号为正，则$Y_0 = 0$， $[X \times Y]_{补} = [X]_{补} \times 0.Y_1Y_2 \cdots Y_n$ **（与上面左侧式子一致）**

当乘数Y的符号为负，则$Y_0 = 1$， $[X \times Y]_{补} = [X]_{补} \times 0.Y_1Y_2 \cdots Y_n - [X]_{补}$ **（与上面右侧式子一致）**

提炼

- 补码乘法不能像原码那样“符号与数值分开”
- 把乘数按补码展开，利用 2^k 的权重合并项
- 合并后出现“相邻两位之差”，这就是 Booth 重编码的来源

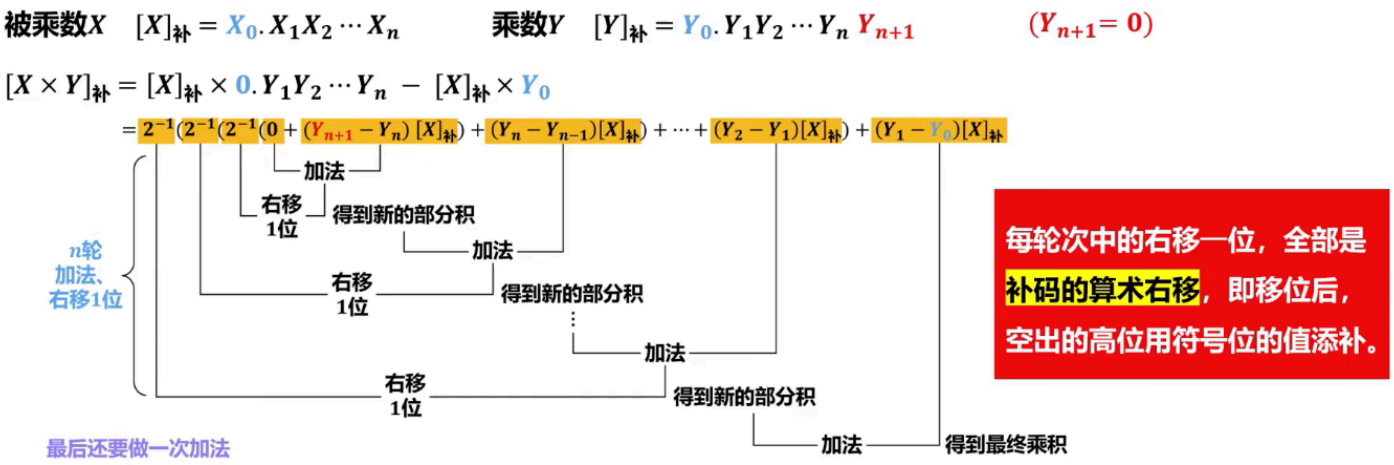
二、依据补码一位乘法运算的进一步推导

2.1 裂项、合并

被乘数X $[X]_{补} = X_0.X_1X_2 \cdots X_n$ **乘数Y** $[Y]_{补} = Y_0.Y_1Y_2 \cdots Y_n$

$$
\begin{aligned}
[X \times Y]_{补} &= [X]_{补} \times 0.Y_1Y_2 \cdots Y_n - [X]_{补} \times Y_0 \\
&= [X]_{补} \times (Y_1 \times 2^{-1} + Y_2 \times 2^{-2} + \cdots + Y_n \times 2^{-n}) - [X]_{补} \times Y_0 \\
&= [X]_{补} \times (-Y_0 + Y_1 \times 2^{-1} + Y_2 \times 2^{-2} + \cdots + Y_n \times 2^{-n}) \\
&= [X]_{补} \times (-Y_0 + (Y_1 - Y_1 \times 2^{-1}) + (Y_2 \times 2^{-1} - Y_2 \times 2^{-2}) + \cdots + (Y_n \times 2^{-(n-1)} - Y_n \times 2^{-n})) \\
&= [X]_{补} \times (-Y_0 + Y_1 - Y_1 \times 2^{-1} + Y_2 \times 2^{-1} - Y_2 \times 2^{-2} + \cdots + Y_n \times 2^{-(n-1)} - Y_n \times 2^{-n}) \\
&= [X]_{补} \times ((Y_1 - Y_0) + 2^{-1}(Y_2 - Y_1) + 2^{-2}(Y_3 - Y_2) + \cdots + 2^{-(n-1)}(Y_n - Y_{n-1}) + 2^{-n}(0 - Y_n)) \\
&= (Y_1 - Y_0)[X]_{补} + 2^{-1}(Y_2 - Y_1)[X]_{补} + 2^{-2}(Y_3 - Y_2)[X]_{补} + \cdots + 2^{-(n-1)}(Y_n - Y_{n-1})[X]_{补} + 2^{-n}(0 - Y_n)[X]_{补} \\
&= (Y_1 - Y_0)[X]_{补} + 2^{-1}((Y_2 - Y_1)[X]_{补} + 2^{-1}(Y_3 - Y_2)[X]_{补} + \cdots + 2^{-(n-1)+1}(Y_n - Y_{n-1})[X]_{补} + 2^{-n+1}(0 - Y_n)[X]_{补}) \\
&= (Y_1 - Y_0)[X]_{补} + 2^{-1}((Y_2 - Y_1)[X]_{补} + 2^{-1}((Y_3 - Y_2)[X]_{补} + \cdots + 2^{-(n-1)+2}(Y_n - Y_{n-1})[X]_{补} + 2^{-n+2}(0 - Y_n)[X]_{补})) \\
&\vdots \\
&= 2^{-1}(2^{-1}(2^{-1}(0 + (0 - Y_n)\,[X]_{补}) + (Y_n - Y_{n-1})[X]_{补}) + \cdots + (Y_2 - Y_1)[X]_{补}) + (Y_1 - Y_0)[X]_{补}
\end{aligned}
$$

2.2 对合并后式子的分析



$Y_{i+1} - Y_i$	操　作
0	部分积 + 0
1	部分积 + $[X]_{补}$
−1	部分积 $-[X]_{补}$ ，相当于 部分积 $+[-X]_{补}$

Booth 重编码表

Q_i	Q_{i-1}	操作
0	0	不加不减
0	1	加 M
1	0	减 M
1	1	不加不减

- Q_i 是当前位，Q_{i-1} 是低位（初始为 0）
- “减 M”用加 $[-M]_{补}$ 实现
- 每步操作后算术右移

易错

- 重编码看的是“当前位与低位”，不是“当前位与高位”
- 10 是减，01 是加，不要记反
- 初始 $Q_{-1} = 0$，不是 1

三、Booth算法的硬件逻辑实现

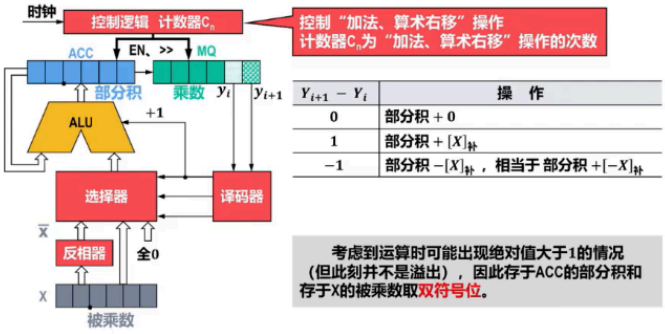
补码一位乘法运算（Booth算法）的硬件逻辑实现

$$[X]_{补} = X_0.X_1X_2\cdots X_n$$

$$[Y]_{补} = Y_0.Y_1Y_2\cdots Y_n\ Y_{n+1}\ （Y_{n+1}\text{为辅助位，初始值为}0）$$

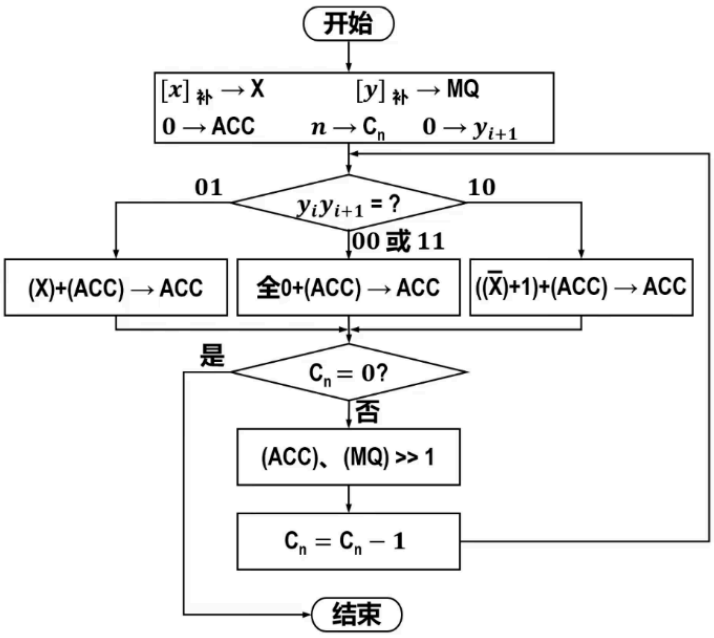
$$[X\times Y]_{补} = [X]_{补}\times 0.Y_1Y_2\cdots Y_n - [X]_{补}\times Y_0$$

$$= 2^{-1}(2^{-1}(2^{-1}(0+(Y_{n+1}-Y_n)[X]_{补})+(Y_n-Y_{n-1})[X]_{补})+\cdots+(Y_2-Y_1)[X]_{补})+(Y_1-Y_0)[X]_{补}$$



$Y_{i+1}-Y_i$	操　作
0	部分积 + 0
1	部分积 + $[X]_{补}$
−1	部分积 $-[X]_{补}$，相当于 部分积 $+[-X]_{补}$

考虑到运算时可能出现绝对值大于1的情况（但此刻并不是溢出），因此存于ACC的部分积和存于X的被乘数取双符号位。



部件

- M：被乘数
- Q：乘数
- Q_{-1}：乘数最低位的前一位，初始 0
- A：部分积，初始 0
- 计数器：循环 n 次

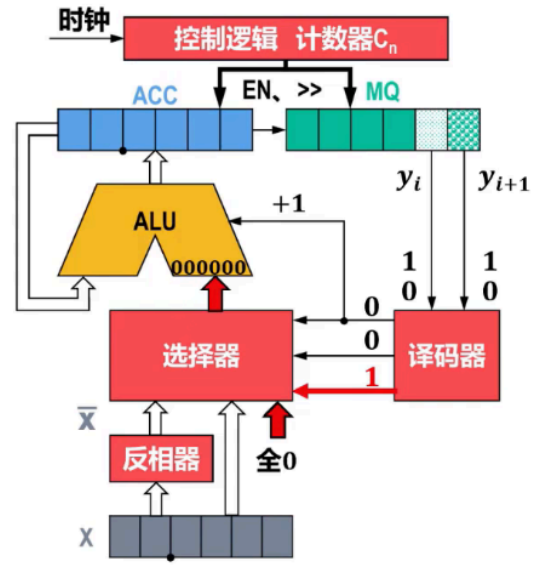
流程

1. 根据 Q_0Q_{-1} 查表，决定 $A \leftarrow A+M$、$A \leftarrow A-M$ 或不变
2. A, Q, Q_{-1} 联合算术右移 1 位
3. 计数器减 1，不为 0 则重复
4. 结束后 A, Q 拼接为乘积

注：采用双符号位，双符号位见 3.5 定点数的加减运算——溢出检测

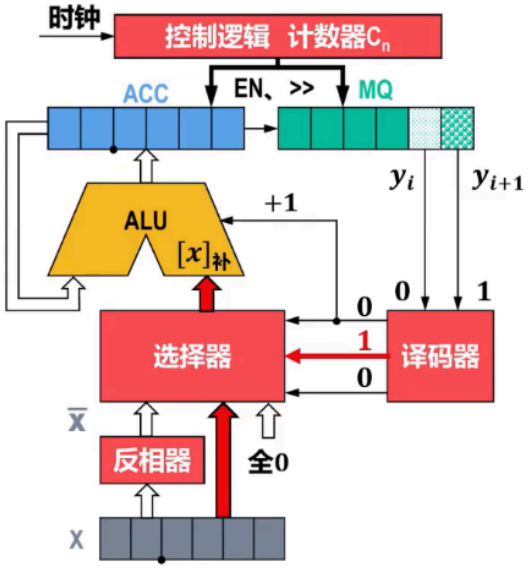
译码器的作用

1. 后一位减前一位为 0



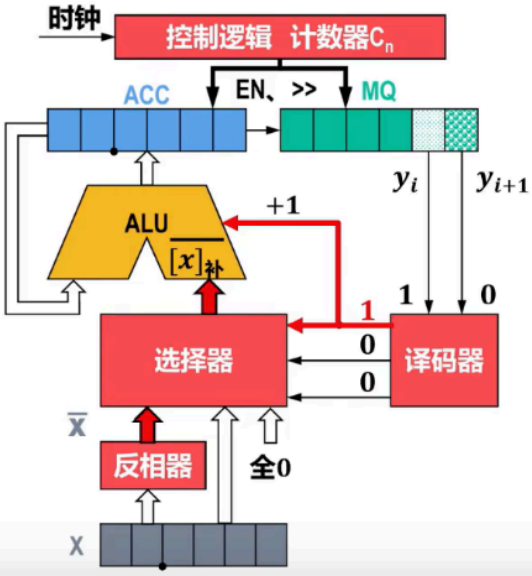
$Y_{i+1} - Y_i$	操　作
0	部分积 + 0
1	部分积 + $[X]_{补}$
−1	部分积 $-[X]_{补}$ ，相当于 部分积 $+[-X]_{补}$

2. 后一位减前一位为 1



$Y_{i+1} - Y_i$	操　作
0	部分积 + 0
1	部分积 + $[X]_{补}$
−1	部分积 $-[X]_{补}$ ，相当于 部分积 $+[-X]_{补}$

3. 后一位减前一位为 -1



$Y_{i+1} - Y_i$	操　作
0	部分积 + 0
1	部分积 + $[X]_{补}$
−1	部分积 $-[X]_{补}$ ，相当于 部分积 $+[-X]_{补}$

易错

- 算术右移时符号位不变，A 的最高位补符号位

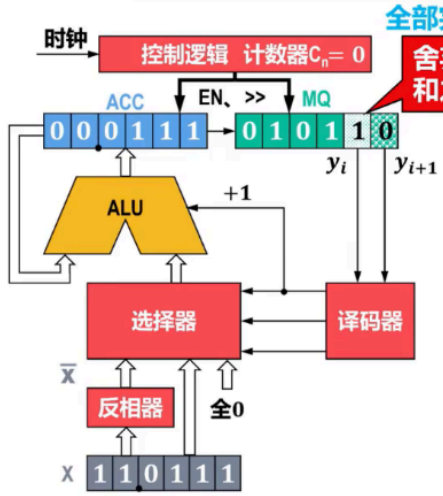
- Q 的最低位进入 Q_{-1}，A 的最低位进入 Q 的最高位
- 双符号位用于容纳中间结果，防止溢出
- Booth 算法对正负乘数都适用，不需要先转原码

四、Booth算法执行示例

设 $x=-0.1001$，$y=-0.1101$，计算 $[x\times y]_{补}$。 $[x]_{补}=11.0111$ $[-x]_{补}=00.1001$ $[y]_{补}=1.0011\,0$

$$[x\times y]_{补}=2^{-1}(2^{-1}(2^{-1}(2^{-1}(0+(0-1)[X]_{补})+(1-1)[X]_{补})+(1-0)[X]_{补})+(0-0)[X]_{补})+(0-1)[X]_{补}$$

$$=2^{-1}(2^{-1}(2^{-1}(2^{-1}(0+[-X]_{补})+0)+[X]_{补})+0)+[-X]_{补}=0.01110101$$



	加数 被乘数	被加数、和 乘积高位	乘数 乘积低位
	X	ACC	MQ
初始状态	110111	000000	100110
$(\overline{X})+1)+(ACC)\rightarrow ACC$	110111	001001	100110
(ACC)、(MQ) >> 1	110111	000100	110011
全0+(ACC) → ACC	110111	000100	110011
(ACC)、(MQ) >> 1	110111	000010	011001
(X)+(ACC) → ACC	110111	111001	011001
(ACC)、(MQ) >> 1	110111	111100	101100
(X)+(ACC) → ACC	110111	111100	101100
(ACC)、(MQ) >> 1	110111	111110	010110
$(\overline{X})+1)+(ACC)\rightarrow ACC$	110111	000111	010110

执行要点

1. 写出 $[M]_{补}$、$[-M]_{补}$
2. $A=0$，Q 装乘数，$Q_{-1}=0$
3. 逐拍查表、加减、算术右移
4. 最后 A,Q 拼接为 $2n$ 位乘积

易错

- 加减后不要忘记右移
- 右移是算术右移，不是逻辑右移
- 结果的符号由补码自然产生，不需要单独异或
- 循环次数等于乘数位数 n

知识关联：补码加减（3.4 定点数的加减运算——补码运算规则）；溢出与双符号位（3.5 定点数的加减运算——溢出检测）；无符号乘法（3.10 无符号数乘法的硬件逻辑实现）；阵列乘法（3.13 无符号阵列乘法器）